

Name:Durga Solanke Roll No: B-47 Practical No -02

Aim:Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

1. What is data ingestion and why is it important in data science? Which Python library is commonly used for data ingestion and cleaning? = Data ingestion is the process of collecting and importing data from various sources (like files, databases, APIs) into a system for analysis. Importance: It is the first step in the data science process. Ensures data is available for analysis. Helps gather data from multiple sources into one place. Enables better decision-making based on collected data. Common Python Library: The most commonly used library is Pandas, which is used for both data ingestion and data cleaning.
2. Why is data cleaning necessary before performing analysis or machine learning? = Data cleaning is necessary because raw data often contains: Missing values Duplicate records Incorrect or inconsistent data Outliers Reasons: Improves accuracy of results Prevents errors in analysis Enhances model performance in machine learning Makes data consistent and reliable Without cleaning, the results of analysis can be misleading or incorrect.
3. How do you load a CSV file into a Pandas DataFrame? = We can load a CSV file using the read_csv() function: import pandas as pd data = pd.read_csv("file_name.csv") print(data.head()) This reads the CSV file and stores it in a DataFrame.
4. What is the difference between head() and tail() functions in Pandas? = head() → Displays the first 5 rows of the dataset tail() → Displays the last 5 rows of the dataset Example: print(data.head()) # First 5 rows print(data.tail()) # Last 5 rows Both functions help quickly inspect data.
5. How can you identify missing values in a dataset? = We can use the following methods: Check missing values print(data.isnull()) Count missing values in each column print(data.isnull().sum()) isnull() → Shows True/False for missing values sum() → Counts total missing values in each column

```
import pandas as pd
# df = pd.read_excel('employee_performance_data.xlsx') # Original line causing FileNotFoundError
# Creating a sample DataFrame for demonstration purposes
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [24, 27, 22, 32, 29, 25, 30, 23, 31, 26],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Performance_Rating': [4, 3, 5, 2, 4, 3, 5, 2, 4, 3]
}
df = pd.DataFrame(data)
print("Step 3: Display the first 5 rows of the DataFrame")
print(df.head()) # By default, shows the first 5 rows
print("\nStep 4: Accessing the 'Name' column")
print(df['Name'])
print("\nStep 5: Selecting 'Name' and 'Performance_Rating' columns")
print(df[['Name', 'Performance_Rating']])
print("\nStep 6: Filtering students with Performance_Rating > 2")
filtered_df = df[df['Performance_Rating'] > 2]
print(filtered_df)
print("\nStep 7: Descriptive statistics")
print(df.describe())
print("\nStep 8: Sorting by 'Performance_Rating' in descending order")
sorted_df = df.sort_values(by='Performance_Rating', ascending=False)
print(sorted_df)
print("\nStep 9: Adding a new column 'Pass/Fail'")
df['Pass/Fail'] = df['Performance_Rating'].apply(lambda x: 'Pass' if x >= 3 else 'Fail') # Adjusted
```

```
print(df)
print("\nStep 10: Handling missing data")
df.loc[2, 'Performance_Rating'] = None # Introducing NaN (missing value) in the 'Performance_Ratin
print(df)
df['Performance_Rating'] = df['Performance_Rating'].fillna(df['Performance_Rating'].mean())
print("\nAfter filling missing 'Performance_Rating' values with mean:")
print(df)
df.to_csv('students_updated.csv', index=False)
print("\nStep 11: Data saved to 'students_updated.csv'")
df['Performance_Rating'] = df['Performance_Rating'].fillna(df['Performance_Rating'].median())
print("\nStep 3: After filling missing 'Performance_Rating' values with median:")
print(df)
print("\nStep 4: Grouping by 'Gender' and calculating the mean 'Performance_Rating'")
grouped_df = df.groupby('Gender')['Performance_Rating'].mean()
print(grouped_df)
print("\nStep 5: Apply a custom function to modify the 'Age' column (e.g., adding 1 year to each a
df['Age'] = df['Age'].apply(lambda x: x + 1)
print(df)
df['Age_in_Months'] = df['Age'] * 12
print("\nStep 6: Added a new column 'Age_in_Months'")
print(df)
print("\nStep 7: Sorting by 'Gender' and then 'Performance_Rating'")
sorted_df = df.sort_values(by=['Gender', 'Performance_Rating'], ascending=[True, False])
print(sorted_df)
df2 = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
'City': ['New York', 'Los Angeles', 'Chicago', 'Houston', 'Phoenix'] })
print("\nStep 8: Merging 'df' with a new DataFrame 'df2' on 'Name' column")
merged_df = pd.merge(df, df2, on='Name', how='inner')
print(merged_df)
print("\nStep 9: Using pivot_table to calculate average Performance_Rating by gender")
pivot_df = pd.pivot_table(df, values='Performance_Rating', index='Gender', aggfunc='mean')
print(pivot_df)
print("\nStep 10: Applying a function to every element using applymap")
df_numbers = df[['Age', 'Performance_Rating']] # Selecting only numerical columns for applymap
df_numbers = df_numbers.map(lambda x: x * 2) # Doubling each value, replaced applymap with map
print(df_numbers)
print("\nStep 11: Reshaping data using melt")
df_melted = pd.melt(df, id_vars=['Name'], value_vars=['Age', 'Performance_Rating'], var_name='Attr
print(df_melted)
```

Step 10: Applying a function to every element using applymap

	Age	Performance_Rating
0	50	8.000000
1	56	6.000000
2	46	6.666667
3	66	4.000000
4	60	8.000000
5	52	6.000000
6	62	10.000000
7	48	4.000000
8	64	8.000000
9	54	6.000000

Step 11: Reshaping data using melt

	Name	Attribute	Value
0	Alice	Age	25.000000
1	Bob	Age	28.000000
2	Charlie	Age	23.000000
3	David	Age	33.000000
4	Eva	Age	30.000000
5	Frank	Age	26.000000
6	Grace	Age	31.000000
7	Heidi	Age	24.000000
8	Ivan	Age	32.000000
9	Judy	Age	27.000000
10	Alice	Performance_Rating	4.000000
11	Bob	Performance_Rating	3.000000
12	Charlie	Performance_Rating	3.333333
13	David	Performance_Rating	2.000000
14	Eva	Performance_Rating	4.000000
15	Frank	Performance_Rating	3.000000
16	Grace	Performance_Rating	5.000000
17	Heidi	Performance_Rating	2.000000
18	Ivan	Performance_Rating	4.000000
19	Judy	Performance_Rating	3.000000

Conclusion Data ingestion and cleaning are essential steps in the data analysis process. Raw data is often incomplete, inconsistent, and contains errors. Through ingestion, data is collected from reliable sources, and through cleaning, issues such as missing values, duplicates, and incorrect data types are resolved. Proper data cleaning improves the accuracy, reliability, and quality of analysis results. Therefore, well-prepared data ensures better decision-making and more meaningful insights in any data-driven project.

